

SQL Foundations



What is SQL ?

Structured Query Language (SQL) is the standard language used to interact with relational databases.

- ✓ It allows users to store, retrieve, update and manage data efficiently through simple commands.
- ✓ It is known for its user-friendly syntax and powerful capabilities, SQL is widely used across industries.

◆ Simple Definition:

- SQL is used to communicate with a database.
- Just like we use English to talk to people, we use SQL to talk to databases.

◆ What Can SQL Do?

SQL helps you:

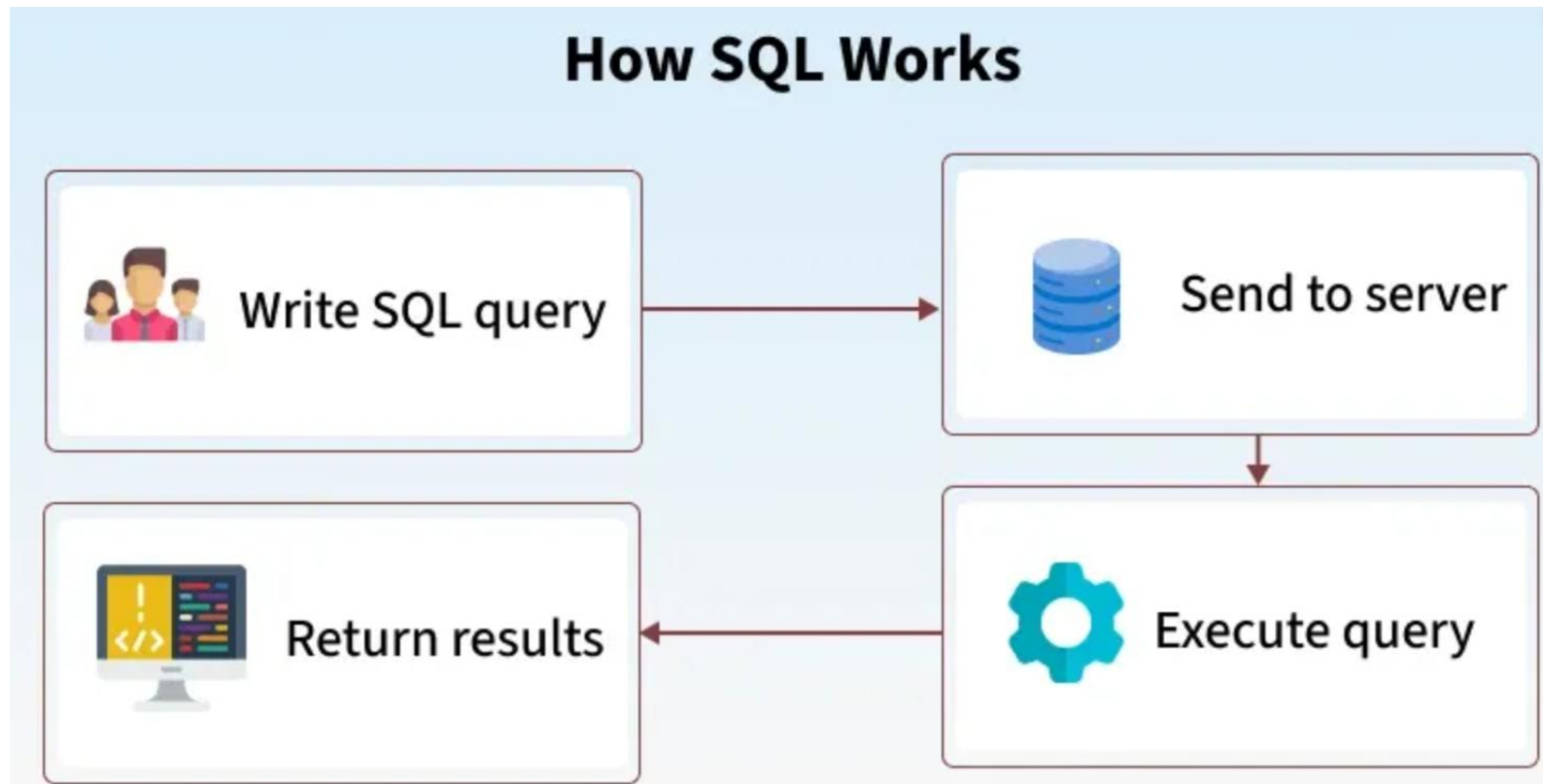
- Create databases and tables
- Insert data
- Read (fetch) data
- Update data
- Delete data
- Control access (security & permissions)

◆ Why SQL is Important?

- Used in Data Engineering
- Used in Data Analysis
- Used in Backend Development
- Core skill for Database Interviews
- Required for working with relational databases.

How does SQL work ?

We interact with databases using SQL queries. DBMS tools like MySQL and SQL Server have their own SQL engine and an interface where users can write and execute SQL queries.



Installation Guide:

Guide attached/shared to the class.

MySQL Cheat sheets/Scripts:

Scripts attached/shared to the class.

Key Components of a SQL System:

- **Databases** : A database is a structured collection of data. It organizes data into **tables**, which are like spreadsheets with rows (records) and columns (fields).
- **Tables**: Each table enforces rules and relationships among its columns for data integrity.
- **Indexes**: Indexes speed up queries by allowing the database to quickly locate data without scanning the entire table.
- **Views**: A view is a virtual table basically a saved SELECT statement you can query like a table.
- **Stored Procedures**: These are pre-written SQL scripts stored inside the database. They can receive inputs, run complex logic and return results boosting performance, reusability and security.
- **Transactions**: A transaction groups multiple SQL operations into a single unit. It ensures **all** changes are applied successfully or none are, preserving data integrity (ACID properties)
- **Security and Permissions**: SQL includes tools to **restrict access**, letting DBAs assign who can do what whether it's accessing tables, executing procedures, or changing structures.
- **Joins**: Joins combine data from multiple tables based on relationships essential for querying across related datasets.

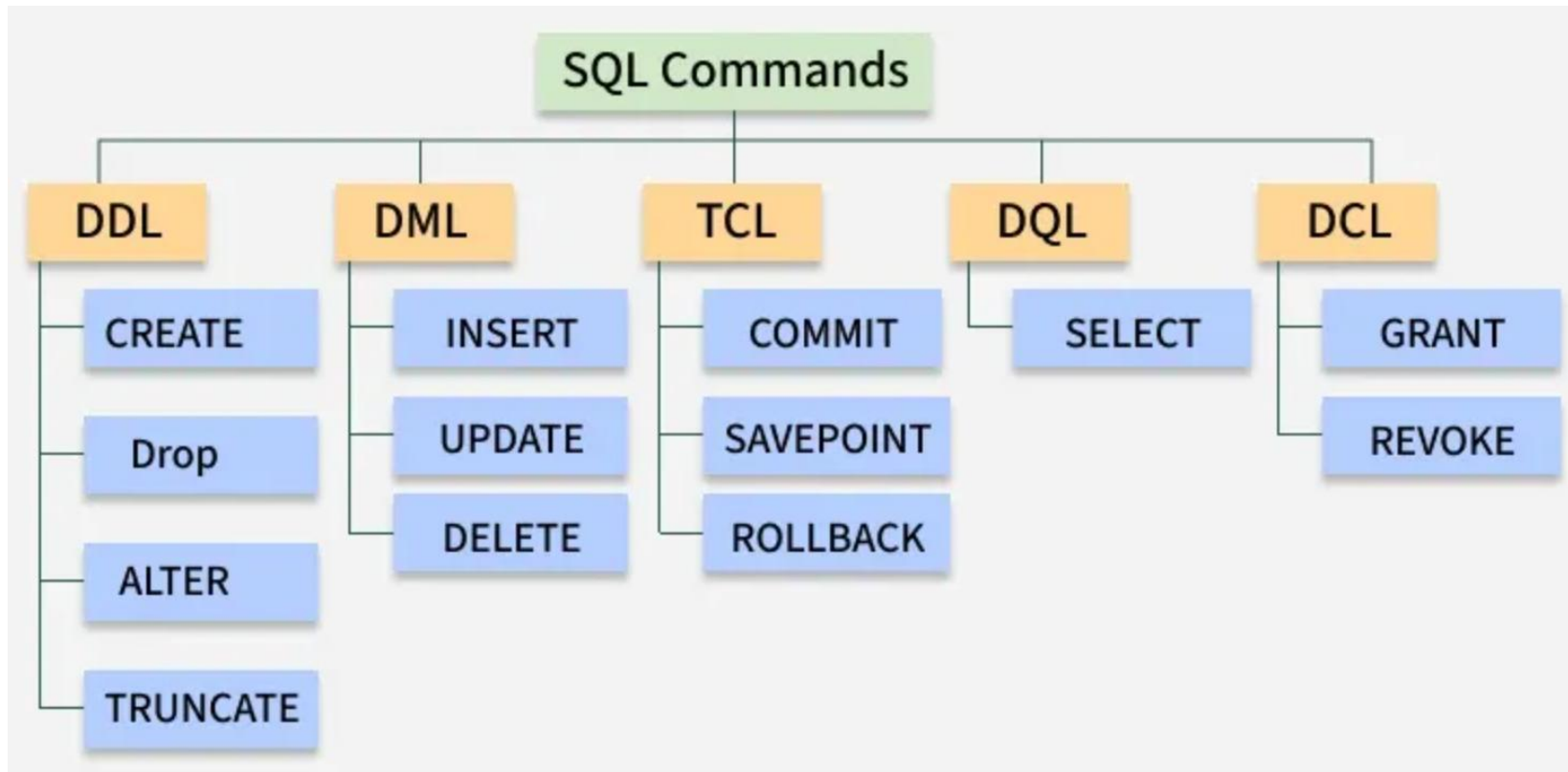
Rules for Writing SQL Queries:

There are certain rules for SQL which would ensure consistency and functionality across databases. By following these rules, queries will be well formed and well executed in any database.

- **End with Semicolon (;):** Each SQL statement must end with a semicolon to execute properly.
- **Case Insensitivity:** SQL keywords (e.g., SELECT, INSERT) are not case-sensitive. However, table or column names may be case-sensitive depending on the DBMS.
- **Whitespace Allowed:** Queries can span multiple lines, but use spaces between keywords and names.
- **Reserved Words:** Avoid using SQL keywords as names. If needed, wrap them in quotes (" ") or backticks (`).
- **Comments:**
 - Single-line: `-- comment`
 - Multi-line: `/* comment */`
- **Data Constraints:** Use **NOT NULL, UNIQUE, PRIMARY KEY, etc.**, to ensure data accuracy.
- **String Values:** Enclose strings in single quotes ('text').
- **Naming Rules:**
 - We Start with a letter
 - We can only use Max 30 characters
 - We only use letters, numbers and underscores (_)

What are Different SQL Commands or Queries?

These commands allow for the creation, manipulation, retrieval and control of data, as well as database structures. SQL commands are categorized based on their specific functionalities:



1. Data Definition Language(DDL):

These SQL commands are used to define, modify or delete the **structure** of database objects such as TABLES, INDEXES and SCHEMAS/DATABASES. Based on the needs of the business, database engineers create and modify database objects using DDL.

The DDL statements are Auto-commit and cannot be rolled-back.

Command	Description
CREATE	Creates a new table, a view on a table, or some other object in the database.
ALTER	Modifies an existing database object, such as a table
DROP	Deletes an entire table, a view of a table, or other objects in the database. Deletes the entire table structure + data
TRUNCATE	Removes all records from a table but keeps the table structure intact. works only on tables (not databases).

CONSTRAINTS:

Constraints in SQL are rules applied to Table columns to enforce data integrity and accuracy. They ensure that the data entered into a database is valid, consistent, and follows the rules you define.

- **NOT NULL:** The column value should not be empty/null
- **UNIQUE :** The column value should be unique(no duplicate value)
- **PRIMARY KEY :** NOT NULL + UNIQUE
- **FOREIGN KEY:** Creates relationship between two tables. Maintains referential integrity
- **CHECK :** Ensures a condition is satisfied (*supported in MySQL 8.0+*).
- **DEFAULT:** Assigns default value if no value is provided.

NOT NULL

```
CREATE TABLE Students (  
  id INT ,  
  name VARCHAR(50) NOT NULL  
);
```

UNIQUE

```
CREATE TABLE Users (  
  id INT ,  
  email VARCHAR(100) UNIQUE  
);
```

PRIMARY KEY

```
CREATE TABLE Employees (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  salary INT  
);
```

FOREIGN KEY

```
CREATE TABLE Departments (  
  dept_id INT PRIMARY KEY,  
  dept_name VARCHAR(50)  
);  
  
CREATE TABLE Employees (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)  
);
```

CHECK

```
CREATE TABLE Employees (  
  id INT PRIMARY KEY,  
  salary INT CHECK (salary > 0)  
);
```

DEFAULT

```
CREATE TABLE Orders (  
  id INT PRIMARY KEY,  
  status VARCHAR(20) DEFAULT 'Pending'  
);
```

KEYS in SQL:

- UNIQUE KEY
- PRIMARY KEY
- FOREIGN KEY
- CANDIDATE KEY
- COMPOSITE KEY
- SURROGATE KEY

UNIQUE KEY:

- Ensures that all values in a column(or combination of columns) are unique.
- Can have one NULL value (in most databases) per column
- You can have multiple unique keys in a table.

id	name	price	categ
1	aa	10	x
2	bb	10	x
3	cc	20	x

PRIMARY KEY:

- Uniquely identifies each record in a table.
- Cannot have NULL values.
- Only ONE primary key per table.

id	name	price	cat
1	aa	10	y
2	bb	10'	y
3	cc	20	y
4	null	10	y

FOREIGNKEY:

- Creates link between 2 tables.
- Ensures that values in a column exist in another table's PRIMARY KEY.

Products				Category		
PK	id	name	price	catid	cat_nam	cat_grp
	1	aa	10	x	food	gro
	2	bb	20	y	shoes	mer
	3	cc	20	z	clothes	mer

CANDIDATE KEY:

- A column or set of columns that uniquely identify a record.
- A Table can have multiple candidate keys.
- Primary Key is chosen from candidate keys.

id	name	email	phone	gender
1	rahul			M
2	priya			F
3	love			M
4	null			

COMPOSITE KEY:

- A Primary Key is made of two or more columns
- Uniquely identifies a record using a combination of columns.

o_id	p_id	amount	o_size	o_location
1	101	1000	XL	CA
1	201	2000	S	Banglore
2	101	1000	XL	TN
2				

SURROGATE KEY:

- An Artificial Key, usually a Auto-Increment number.
- Used when Natural Keys(eg: *order_id*: which has business meaning) are inconvenient or too long.
- Example: "Order_id" is Natural key, hence added new column "Key"

order_id	name	price	size	can	key
nhiswoclks	Iphone	2000	s	n	1
xhqjkkxjklck	Samsung	2100	s	n	2
dljwcljcjel	Headset	200	xs	y	3

SQL CREATE TABLE

The CREATE TABLE statement in SQL is used to define a new table in a database. It specifies the table name, column names, and their data types. This command lays the foundation for storing and organizing data.

Syntax:

```
CREATE table table_name  
(  
  Column1 datatype (size),  
  column2 datatype (size),  
  .  
  .  
  columnN datatype(size)  
);
```

- **table_name:** The name you assign to the new table.
- **column1, column2, ... :** The names of the columns in the table.
- **datatype(size):** Defines the data type and size of each column.

Example: Create a Customer Table

We create a Customer table that stores customer data. We will define various columns such as CustomerID, CustomerName, Country, Age, and Phone with appropriate data types and constraints.

Query:

```
CREATE TABLE Customer(  
    CustomerID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Country VARCHAR(50),  
    Age INT CHECK (Age >= 0 AND Age <= 99),  
    Phone int(10)  
);
```

Output :

Customer id	FirstName	LastName	Country	Age	Phone
Empty					

Explanation:

- CustomerID is an integer and serves as the PRIMARY KEY, ensuring each record is unique.
- FirstName, LastName, and Country are VARCHAR fields to store variable-length text.
- Age has a CHECK constraint, ensuring it's within the range of 0 to 99.
- Phone is an integer field, although in real scenarios, a VARCHAR would often be used for storing phone numbers to allow for leading zeros and formatting.

Create Table from Existing Table :

We can also create a new table based on the structure (and optionally the data) of an existing table. The CREATE TABLE AS SELECT command allows us to duplicate an entire table or select specific columns to form a new one. The following query creates a new table called SubTable that contains CustomerID and CustomerName from the existing Customer table.

Syntax:

```
CREATE TABLE new_table_name AS  
SELECT column1, column2, ...  
FROM existing_table_name  
WHERE ...;
```

In this example, we create a new table SubTable that contains just the CustomerID and CustomerName columns from the Customer table. This method is useful for creating backups or performing quick data migrations.

Query :

```
CREATE TABLE SubTable AS  
SELECT CustomerID, CustomerName  
FROM customer;
```

Output :

SubTable

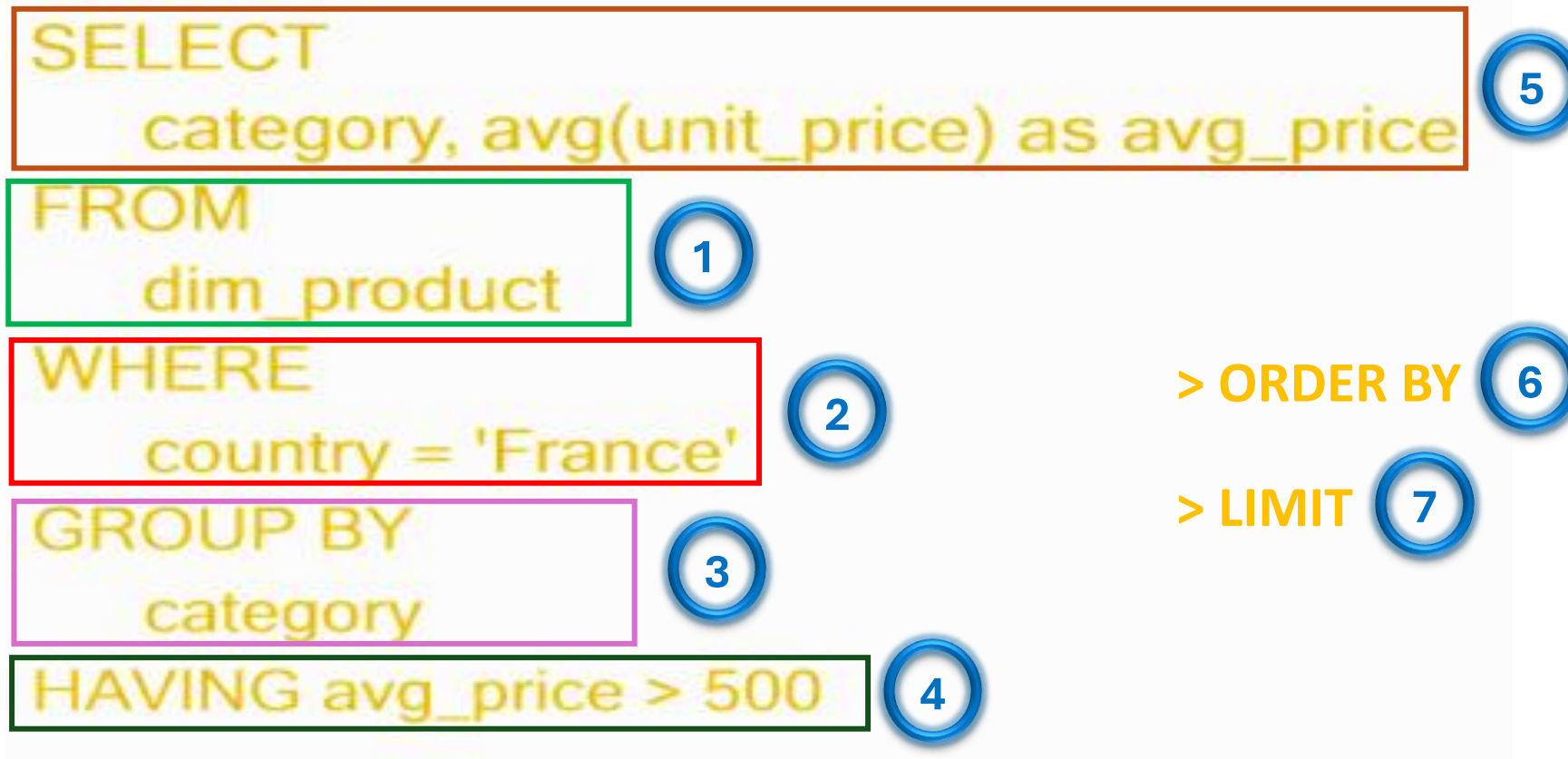
CustomerID	CustomerName
1	Shubham
2	Aman
3	Naveen
4	Aditya

2. Data Query Language(DQL):

Data retrieval instructions are written in the data query language (DQL), which is used to access relational databases. The SELECT command is used by software programs to filter and return particular results from a SQL table.

Command	Description
SELECT	Read/Fetch the data in the table(s)

Query Execution Flow:



Interview Tip:

- The HAVING section in the above query is referring to a field “avg_price” which is not a column in the table “dim_product”. This “avg_price” is not even executed yet as it is at 4th Execution step before its calculation at step SELECT. However, This will work only in MySQL but will not be the case in other SQL softwares like MS SQL etc. This is the power of MySQL.
- For any other SQL software, you need to write like this: ***HAVING avg(unit_price) > 500***

SQL SELECT Query

The SQL SELECT statement retrieves data from one or more tables and returns it as a tabular result set of rows and columns. You can fetch all columns with * or choose specific columns, and narrow results using WHERE filters. Queries can be ordered with ORDER BY, grouped and aggregated with GROUP BY/HAVING, and combined across tables using JOINS.

Syntax:

```
SELECT column1,column2.... FROM table_name;
```

- column1, column2: columns you want to retrieve.
- table_name: name of the table you're querying.

Example 1: Select Specific Columns

In this example, we will demonstrate how to retrieve specific columns from the Customer table. Here we will fetch only CustomerName and LastName for each record.

Query :

```
SELECT CustomerName, LastName  
FROM Customer;
```

Example 2: Select All Columns

In this example, we will fetch all the fields from table Customer:

Query:

```
SELECT * FROM Customer;
```

Example 3: SELECT Statement with WHERE Clause

Suppose we want to see table values with specific conditions then WHERE CLAUSE is used with select statement. In this example, filter customers who are 21 years old.

Query:

```
SELECT CustomerName  
FROM Customer  
where Age = '21';
```

Example 4: SELECT with GROUP BY Clause

In this example, we will use SELECT statement with GROUP BY Clause to group rows and perform aggregation. Here, we will count the number of customers from each country.

Query:

```
SELECT Country, COUNT(*) AS customer_count  
FROM Customer  
GROUP BY Country;
```

Example 5: SELECT with DISTINCT Clause

In this example, we will use DISTINCT keyword to return only unique values from a column. Here, we will fetch unique countries from the Customer table.

Query:

```
SELECT DISTINCT Country  
FROM Customer;
```

Example 6: SELECT Statement with HAVING Clause

The HAVING clause is used to filter results after applying GROUP BY. In this example, we will find countries that have 2 or more customers in the Customer table.

Query:

```
SELECT Country, COUNT(*) AS  
customer_count  
FROM Customer  
GROUP BY Country  
HAVING COUNT(*) >= 2;
```

Example 7: SELECT Statement with ORDER BY clause

In this example, we will use SELECT Statement with ORDER BY clause. Here, Sort results by Age in descending order.

Query:

```
SELECT * FROM Customer ORDER BY Age DESC;
```

SQL - WHERE Clause

The SQL WHERE clause filters rows based on one or more conditions, so your query returns (or modifies) only the records that match. It's used across SELECT, UPDATE, and DELETE statements, and works with data from a single table or from multiple tables after joins

Syntax:

```
SELECT column1, column2  
FROM table_name  
WHERE column_name operator value;
```

- **column1, column2:** Columns you want to retrieve.
- **table_name:** Table you are querying from.
- **operator:** Comparison logic (e.g., =, <, >, LIKE).
- **value:** The value or pattern to filter against.

Example: First, we will create a demo SQL database and table, on which we will use the WHERE Clause command.

EmpID	Name	Department	Salary
1	John Smith	HR	45000
2	Emma Brown	IT	60000
3	David Lee	Sales	52000

Query:

```
SELECT Name, Department, Salary  
FROM Employees  
WHERE Salary > 50000;
```

Name	Department	Salary
Emma Brown	IT	60000
David Lee	Sales	52000

Examples of WHERE Clause

We will create a basic employee table structure in SQL for performing all the where clause operation.

EmpID	Name	Country	Age	Mob
1	John	USA	23	738479734
2	Liam	Australia	21	436789555
3	Noah	Canada	24	34873847
4	Oliver	UK	21	328440934
5	Lucas	Spain	22	73248679

Example 1: Where Clause with Logical Operators

To fetch records of Employee with age equal to 24.

Query:

```
SELECT * FROM Emp1 WHERE Age=24;
```

Output:

EmpID	Name	Country	Age	Mob
3	Noah	Canada	24	34873847

Example 2: WHERE with Comparison Operators

To fetch the EmpID, Name and Country of Employees with Age greater than 21.

Query: `SELECT EmpID, Name, Country FROM Emp1 WHERE Age > 21;`

Output:

EmpID	Name	Country
1	John	USA
3	Noah	Canada
5	Lucas	Spain

Example 3: Where Clause with BETWEEN Operator

The BETWEEN operator is used to filter records within a specified range, and it includes both the start and end values. In this example, we want to find employees whose age is between 22 and 24, including both 22 and 24.

Query:

```
SELECT * FROM Emp1  
WHERE Age BETWEEN 22 AND 24;
```

Output:

EmpID	Name	Country	Age	Mob
1	John	USA	23	738479734
3	Noah	Canada	24	34873847
5	Lucas	Spain	22	73248679

Example 4: Where Clause with LIKE Operator

It is used to fetch filtered data by searching for a particular pattern in the where clause. In this example we want to find records of Employees where Name starts with the letter. The '%' (wildcard) signifies the later characters here which can be of any length and value.

Query:

```
SELECT *  
FROM Employee  
WHERE Name LIKE 'L%';
```

Output:

EmpID	Name	Country	Age	Mob
2	Liam	Australia	21	436789555
5	Lucas	Spain	22	73248679

Example 5: Where Clause with IN Operator

It is used to fetch the filtered data same as fetched by '=' operator just the difference is that here we can specify multiple values for which we can get the result set. Here we want to find the Names of Employees where Age is 21 or 23.

Query:

```
SELECT Name FROM Emp1 WHERE Age IN (21,23);
```

Output:

Name
John
Liam
Oliver

Operators Used in WHERE Clause

Operator	Description
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
=	Equal to
<>	Not equal to
BETWEEN	In an inclusive range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

SQL Joins

SQL joins are fundamental tools for combining data from multiple tables in relational databases.

Types of SQL Joins : Let us visualize how each join type operates:

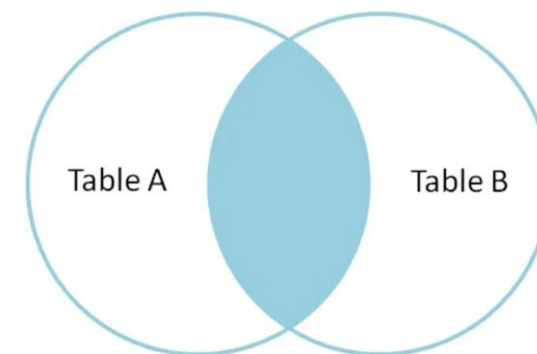
1. INNER JOIN / JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN / UNION
5. SELF JOIN
6. CROSS JOIN

INNER JOIN:

The INNER JOIN keyword selects all rows from both the tables as long as the condition is satisfied. This keyword will create the result set by combining all rows from both the tables where the condition satisfies i.e value of the common field will be the same.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,.... FROM table1  
INNER JOIN  
table2 ON table1.matching_column = table2.matching_column;
```



Query:

```
SELECT * FROM orders o INNER JOIN customers c ON o.cust_id = c.id;
```

Orders

o_id	cust_id	price
1	101	1000
2	201	1100
3	501	1200

customers

id	name	email
101	nag	aa@gmail.com
201	chiru	bb@gmail.com
301	venki	cc@gmail.com



OUTPUT

o_id	cust_id	price	id	name	email
1	101	1000	101	nag	aa@gmail.com
2	201	1100	201	chiru	bb@gmail.com

LEFT JOIN:

A LEFT JOIN returns all rows from the left table, along with matching rows from the right table. If there is no match, NULL values are returned for columns from the right table.

Syntax:

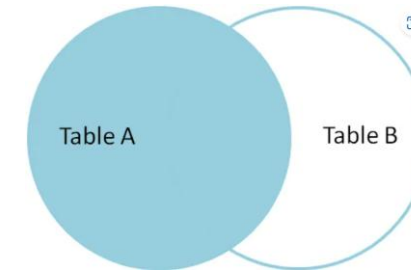
```
SELECT table1.column1,table1.column2,table2.column1,...FROM table1
```

```
LEFT JOIN table2
```

```
ON table1.matching_column = table2.matching_column;
```

Query:

```
SELECT * FROM orders o LEFT JOIN customers c ON o.cust_id = c.id;
```



Orders			customers		
o_id	cust_id	price	id	name	email
1	101	1000	101	nag	aa@gmail.com
2	201	1100	201	chiru	bb@gmail.com
3	501	1200	301	venki	cc@gmail.com



OUTPUT					
o_id	cust_id	price	id	name	email
1	101	1000	101	nag	aa@gmail.com
2	201	1100	201	chiru	bb@gmail.com
3	501	1200	NULL	NULL	NULL

RIGHT JOIN:

A RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. It is very similar to LEFT JOIN for the rows for which there is no matching row on the left side, the result-set will contain null.

Syntax:

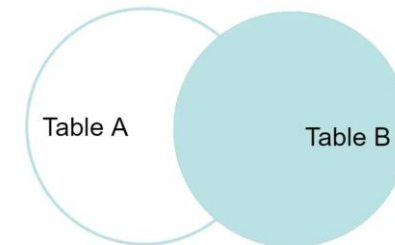
```
SELECT table1.column1,table1.column2,table2.column1,...FROM table1
```

```
RIGHT JOIN table2
```

```
ON table1.matching_column = table2.matching_column;
```

Query:

```
SELECT * FROM orders o RIGHT JOIN customers c ON o.cust_id = c.id;
```



Orders			customers		
o_id	cust_id	price	id	name	email
1	101	1000	101	nag	aa@gmail.com
2	201	1100	201	chiru	bb@gmail.com
3	501	1200	301	venki	cc@gmail.com



OUTPUT					
o_id	cust_id	price	id	name	email
1	101	1000	101	nag	aa@gmail.com
2	201	1100	201	chiru	bb@gmail.com
NULL	NULL	NULL	301	venki	cc@gmail.com

UNION:

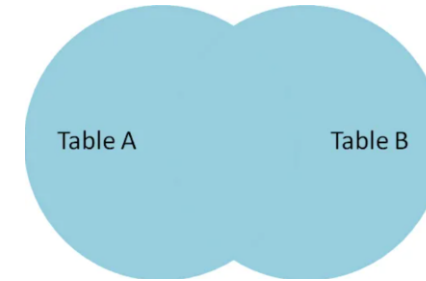
UNION/FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.

Query:

```
SELECT * FROM orders o LEFT JOIN customers c ON o.cust_id = c.id
```

UNION

```
SELECT * FROM orders o RIGHT JOIN customers c ON o.cust_id = c.id
```



Orders			customers		
o_id	cust_id	price	id	name	email
1	101	1000	101	nag	aa@gmail.com
2	201	1100	201	chiru	bb@gmail.com
3	501	1200	301	venki	cc@gmail.com



OUTPUT					
o_id	cust_id	price	id	name	email
1	101	1000	101	nag	aa@gmail.com
2	201	1100	201	chiru	bb@gmail.com
3	501	1200	NULL	NULL	NULL
NULL	NULL	NULL	301	venki	cc@gmail.com

SELF JOIN: A **SELF JOIN** is a regular join where a table is joined **with itself**.

It is mainly used when:

- Comparing rows within the same table
- Finding hierarchical relationships (like Employee–Manager)

Query:

```
SELECT e.emp_name AS Employee, m.emp_name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.emp_id;
```

Example: Employee–Manager Relationship

Employees Table		
emp_id	emp_name	manager_id
1	Ravi	NULL
2	Anu	1
3	Kiran	1
4	Meena	2

Here:

- Ravi (1) is a manager
- Anu (2) reports to Ravi (1)
- Meena (4) reports to Anu (2)



✓ Output	
Employee	Manager
Ravi	NULL
Anu	Ravi
Kiran	Ravi
Meena	Anu

CROSS JOIN:

SQL CROSS JOIN creates a Cartesian product of two tables, meaning it returns every possible combination of rows from both tables. It does not use a join condition, and the total number of rows in the result is the number of rows in the first table multiplied by the number of rows in the second table. It is useful for generating all combinations of data.

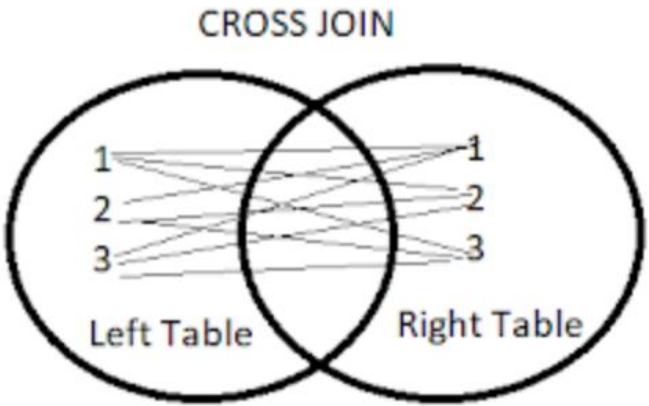
Explanation: CROSS JOIN combines every row from the Customer table with every row from Orders table. Since there are 2 customers and 2 orders, result contains $2 \times 2 = 4$ rows.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....FROM table1  
CROSS JOIN table2;
```

Query:

```
SELECT * FROM orders o CROSS JOIN customers c;
```



Orders		
o_id	cust_id	price
1	101	1000
2	201	1100
3	501	1200

customers		
id	name	email
101	nag	aa@gmail.com
201	chiru	bb@gmail.com
301	venki	cc@gmail.com



OUTPUT

o_id	cust_id	price	id	name	email
3	501	1200	101	nag	aa@gmail.com
2	201	1100	101	nag	aa@gmail.com
1	101	1000	101	nag	aa@gmail.com
3	501	1200	201	chiru	bb@gmail.com
2	201	1100	201	chiru	bb@gmail.com
1	101	1000	201	chiru	bb@gmail.com
3	501	1200	301	venki	cc@gmail.com
2	201	1100	301	venki	cc@gmail.com
1	101	1000	301	venki	cc@gmail.com

SQL | String functions

SQL string functions help manipulate and format text data efficiently. They are widely used for cleaning, comparing, and extracting meaningful information from textual fields.

Useful for handling names, addresses, and other text-based data.

Essential for organizing, analyzing, and improving data quality.

SQL String Functions

String functions are used to perform an operation on input string and return an output string. Below are some of the most commonly used SQL string functions:

1. CONCAT()

The CONCAT() function is used to concatenate (combine) two or more strings into one string. It is useful when we want to merge fields like first and last names into a full name.

Query:

```
SELECT CONCAT('John', ' ', 'Doe') AS FullName;
```

Output:

```
John Doe
```

2. CHAR_LENGTH() / CHARACTER_LENGTH()

The CHAR_LENGTH() or LENGTH() function returns the length of a string in characters. It's essential for validating or manipulating text data, especially when you need to know how many characters a string contains.

Query:

```
SELECT CHAR_LENGTH('Hello') AS StringLength;
```

Output:

```
5
```

3. UPPER() and LOWER()

These UPPER() and LOWER() functions convert the text to uppercase or lowercase, respectively. They are useful for normalizing the case of text in a database.

Query:

```
SELECT UPPER('hello') AS UpperCase;  
SELECT LOWER('HELLO') AS LowerCase;
```

Output:

```
HELLO  
hello
```

4. LENGTH()

LENGTH() returns the length of a string in bytes. This can be useful for working with multi-byte character sets.

Query:

```
SELECT LENGTH('Hello') AS LengthInBytes;
```

Output:

```
5
```

5. REPLACE()

The REPLACE() function replaces occurrences of a substring within a string with another substring. This is useful for cleaning up data, such as replacing invalid characters or formatting errors.

Query:

```
SELECT REPLACE('Hello World', 'World', 'SQL') AS UpdatedString;
```

Output:

```
Hello SQL
```

6. SUBSTRING() / SUBSTR()

The SUBSTRING() (or SUBSTR()) function is used to extract a substring from a string, starting from a specified position. It is especially useful when we need to extract a specific part of a string, like extracting the domain from an email address.

Query:

```
SELECT SUBSTRING('Hello World', 1, 5) AS SubStringExample;
```

Output:

```
Hello
```

7. LEFT() and RIGHT()

The LEFT() and RIGHT() functions allow you to extract a specified number of characters from the left or right side of a string, respectively. It is used for truncating strings for display.

Query:

```
SELECT LEFT('Hello World', 5) AS LeftString;  
SELECT RIGHT('Hello World', 5) AS RightString;
```

Output:

```
Hello  
World
```


8. INSTR()

The INSTR() function is used to find the position of the first occurrence of a substring within a string. It returns the position (1-based index) of the substring. If the substring is not found, it returns 0. This function is particularly useful for locating specific characters or substrings in text data.

Query:

```
SELECT INSTR('Hello World', 'World') AS SubstringPosition;
```

Output:

```
7
```

9. TRIM()

The TRIM() function removes leading and trailing spaces (or other specified characters) from a string. By default, it trims spaces but can also remove specific characters using TRIM(character FROM string). This is helpful for cleaning text data, such as user inputs or database records.

Query:

```
SELECT TRIM(' ' FROM ' Hello World ') AS TrimmedString;
```

Output:

```
Hello World
```

10. REVERSE()

The REVERSE() function reverses the characters in a string. It's useful in situations where we need to process data backward, such as for password validation or certain pattern matching.

Query:

```
SELECT REVERSE('Hello') AS ReversedString;
```

Output:

```
olleH
```

Advanced String Functions

In SQL, beyond the basic string functions, there are several advanced string functions that can help you manipulate and process string data more effectively. These are the some additional SQL Functions.

11. ASCII()

The ASCII() function returns the ASCII value of a single character. This is helpful when we need to find the numeric code corresponding to a character, often used in encoding and decoding text.

Query:

```
SELECT ascii('t');
```

Output:

```
116
```

12. CONCAT_WS()

CONCAT_WS() stands for "Concatenate With Separator." It allows us to join multiple strings with a specific separator between them. This is ideal when we need to merge columns like first name and last name with a custom separator.

Query:

```
SELECT CONCAT_WS('_', 'V', 'CUBE');
```

Output:

```
V_CUBE
```

13. FIND_IN_SET()

The FIND_IN_SET() function returns the position of a value within a comma-separated list. This is especially useful for finding out where an element exists in a string of values (e.g., tags, categories).

Query:

```
SELECT FIND_IN_SET('b', 'a, b, c, d, e, f');
```

Output:

```
2
```

14. FORMAT()

The FORMAT() function is used to format a number as a string in a specific way, often with commas for thousands or with a specific number of decimal places. It is handy when you need to display numbers in a user-friendly format.

Query:

```
SELECT FORMAT(0.981 * 100, 'N2') + '%' AS PercentageOutput;
```

Output:

```
'98.10%'
```

15. LCASE()

The LCASE() function converts all characters in a string to lowercase. It helps standardize text data, especially when comparing strings in a case-insensitive way.

Query:

```
SELECT LCASE ("VCUBE To Learn");
```

Output:

```
vcube to learn
```

16. LOCATE()

LOCATE() allows you to find the nth occurrence of a substring in a string. This is especially useful when you need to locate a specific substring based on its position.

Query:

```
SELECT LOCATE('for', 'geeksforgeeks', 1);
```

Output:

6

17. REPEAT()

The REPEAT() function repeats a string a specified number of times. It's useful when you need to duplicate a string or pattern for certain operations.

Query:

```
SELECT REPEAT('vcube', 2);
```

Output:

vcubevcube

SPACE()

The SPACE() function generates a string consisting of a specified number of spaces. This is useful when you need to format output or create padding in your queries.

Query:

```
SELECT SPACE(7);
```

Output:

```
'      '
```

SQL | Date Functions

SQL Date Functions are built-in tools used to handle, modify, and analyze date/time values in a database. They help perform tasks like retrieving current dates, calculating differences, and formatting results effectively.

- Extract specific parts of a date (year, month, day).
- Format dates for user-friendly display.
- Track trends, deadlines, or schedules in business processes.

Let's use the sales table as the base for demonstrating each of the SQL Date Functions mentioned. Here's the structure of the sales table:

sale_id	product_name	sale_date	sale_amount
1	Widget A	2024-08-01	100
2	Widget B	2024-08-05	150
3	Widget C	2024-08-07	200
4	Widget A	2024-08-10	120
5	Widget B	2024-08-15	180
6	Widget C	2024-08-20	250

1. NOW()

The NOW() function retrieves the server's current date and time, making it useful for capturing exact event moments such as transaction timestamps, as well as for logging and comparing time-based records.

Query:

```
SELECT NOW() AS current_datetime;
```

Output:

current_datetime
2025-11-20 09:33:33

- Returns the current system date and time.
- Shows both date and time in one value.

2. CURDATE()

The CURDATE() function returns today's date in YYYY-MM-DD format and is useful when only the current date is needed, especially for reporting or filtering records by date.

Query:

```
SELECT CURDATE() AS current_date;
```

Output:

current_date
2025-11-20

- Returns today's date only.
- Time part is not included.

3. CURTIME()

The CURTIME() function returns the current time in HH:MM:SS format and is useful for time-based operations, such as scheduling or performing precise time comparisons.

Query:

```
SELECT CURTIME() AS current_time;
```

Output:

current_time
09:37:23

- Returns the current system time.
- Shows hours, minutes, and seconds only.

4. DATE()

The DATE() function extracts only the date from a date or datetime value, making it useful for situations where the time component should be ignored, such as date-only comparisons or aggregations.

Query:

```
SELECT sale_id, product_name,  
       DATE(sale_date) AS sale_date_only  
FROM sales;
```

Output

sale_id	product_name	sale_date_only
1	Widget A	2024-08-01
2	Widget B	2024-08-05
3	Widget C	2024-08-07
4	Widget A	2024-08-10
5	Widget B	2024-08-15
6	Widget C	2024-08-20

- Extracts only the date part from sale_date.
- Removes any time portion if present.

5. EXTRACT()

The EXTRACT() function retrieves a specific part of a date such as the year, month, or day, making it useful for grouping, filtering, or performing time-based analysis including year-over-year reports.

Query:

```
SELECT sale_id, product_name,  
       EXTRACT(YEAR FROM sale_date)  
       AS sale_year FROM sales;
```

Output:

sale_id	product_name	sale_year
1	Widget A	2024
2	Widget B	2024
3	Widget C	2024
4	Widget A	2024
5	Widget B	2024
6	Widget C	2024

- Pulls out the year from the sale date.
- Useful for grouping or filtering by year.

6. DATE_ADD()

The DATE_ADD() function adds a chosen time interval such as days, months, or years to a date, making it useful for calculating future dates and simplifying planning or scheduling tasks.

Query:

```
SELECT sale_id, product_name,  
DATE_ADD(sale_date, INTERVAL 7 DAY) AS sale_date_plus_7_days  
FROM sales;
```

Output:

sale_id	product_name	sale_date_plus_7_days
1	Widget A	2024-08-08
2	Widget B	2024-08-12
3	Widget C	2024-08-14
4	Widget A	2024-08-17
5	Widget B	2024-08-22
6	Widget C	2024-08-27

- Adds 7 days to the sale date.
- Returns the new future date.

7. DATE_SUB()

The DATE_SUB() function subtracts a chosen time interval from a date, making it useful for determining past dates and performing retrospective data analysis.

Query:

```
SELECT sale_id, product_name,  
DATE_SUB(sale_date, INTERVAL 3 DAY)  
AS sale_date_minus_3_days  
FROM sales;
```

Output:

sale_id	product_name	sale_date_minus_3_days
1	Widget A	2024-07-29
2	Widget B	2024-08-02
3	Widget C	2024-08-04
4	Widget A	2024-08-07
5	Widget B	2024-08-12
6	Widget C	2024-08-17

- Subtracts 3 days from the sale date.
- Returns an earlier date.

8. DATEDIFF()

The DATEDIFF() function returns the number of days between two dates, making it useful for calculating durations such as deadlines or overdue periods. In this case, it shows how many days remain from each sale date until August 15, 2024.

Query:

```
SELECT sale_id, product_name, sale_date,  
DATEDIFF('2024-08-15', sale_date) AS days_until_aug15  
FROM sales;
```

Output:

sale_id	product_name	sale_date	days_until_aug15
1	Widget A	2024-08-01	14
2	Widget B	2024-08-05	10
3	Widget C	2024-08-07	8
4	Widget A	2024-08-10	5
5	Widget B	2024-08-15	0
6	Widget C	2024-08-20	-5

- Calculates days between two dates.
- Shows how many days until 15-Aug-2024.

9. DATE_FORMAT()

The DATE_FORMAT() function formats a date using a specified pattern, allowing customized output such as full day or month names, and is useful for making reports clearer and more readable.

Query:

```
SELECT sale_id, product_name,  
       DATE_FORMAT(sale_date, '%W, %M %d, %Y')  
       AS formatted_sale_date FROM sales;
```

Output:

sale_id	product_name	formatted_sale_date
1	Widget A	Thursday, August 01, 2024
2	Widget B	Monday, August 05, 2024
3	Widget C	Wednesday, August 07, 2024
4	Widget A	Saturday, August 10, 2024
5	Widget B	Thursday, August 15, 2024
6	Widget C	Tuesday, August 20, 2024

- Converts date into a readable text format.
- Displays weekday, month name, day, and year.

10. ADDDATE()

The ADDDATE() function adds a specified time interval to a date. It is useful for calculating future or past dates based on a given date.

Query:

```
SELECT sale_id, product_name,  
       ADDDATE(sale_date, 10)  
       AS sale_date_plus_10_days  
FROM sales;
```

Output:

sale_id	product_name	sale_date_plus_10_days
1	Widget A	2024-08-11
2	Widget B	2024-08-15
3	Widget C	2024-08-17
4	Widget A	2024-08-20
5	Widget B	2024-08-25
6	Widget C	2024-08-30

- Adds 10 days to the sale date.
- Same as DATE_ADD but without INTERVAL.

11. ADDTIME()

The ADDTIME() function adds a specified time interval to a time or datetime value. It is useful for adjusting times by adding hours, minutes or seconds.

Query:

```
SELECT sale_id, product_name,  
ADDTIME('10:30:00', '02:30:00') AS sale_time_plus_2hrs_30min  
FROM sales;
```

Output:

sale_id	product_name	sale_time_plus_2hrs_30min
1	Widget A	13:00:00
2	Widget B	13:00:00
3	Widget C	13:00:00
4	Widget A	13:00:00
5	Widget B	13:00:00
6	Widget C	13:00:00

- Adds two time values together.
- Returns the new time after addition.

SQL Count() Function

The SQL COUNT() function is used to return the number of rows or non-null values in a column. It helps summarize data in reports and analytics.

- Counts all rows or rows with a condition
- Counts unique values using COUNT(DISTINCT column)
- Commonly used with GROUP BY and HAVING for grouped results

Syntax:

```
COUNT(expression)
```

1. Count all rows:

```
SELECT COUNT(*) FROM table_name;
```

2. Count distinct values in a column:

```
SELECT COUNT(DISTINCT column_name) FROM table_name;
```

Example: First, we create a demo SQL database and table, on which we will use the COUNT() functions.

EmplID	Name	Department
101	John	Sales
102	Emily	HR
103	Michael	Sales

Query:

```
SELECT COUNT(*) AS TotalEmployees FROM Employee;  
SELECT COUNT(DISTINCT Department) AS UniqueDepartments  
FROM Employee;
```

Output:

TotalEmployees
3
UniqueDepartments
2

Examples of SQL Count Function

CustomerID	CustomerName	ContactName	Age	Country	City
1	George	John	28	Spain	Madrid
2	Ana Trujillo	Ana Trujillo	25	Mexico	Mexico City
3	Gabriel	Laura	40	Spain	Barcelona
4	Thomas Hardy	Thomas Hardy	35	Germany	Berlin
5	Rachel	Maria	45	Spain	Seville
6	Kaushik	Kaushik	30	India	Mumbai
7	Maria Gonzalez	Jose	33	Spain	Barcelona
8	Antonio Moreno	Antonio	50	Mexico	Guadalajara
9	Bheem	Sumit	29	India	Delhi

Count Rows That Match a Condition Using COUNT() with CASE WHEN

Query:

```
SELECT COUNT(CASE WHEN Age > 30 THEN 1 ELSE NULL END) AS Adults  
FROM Customers;
```

Adults
5

Count Rows in Groups Using COUNT() with GROUP BY

Query:

```
SELECT Country, COUNT(*) AS CustomerCount  
FROM Customers  
GROUP BY Country;
```

Output:

Country	CustomerCount
Germany	1
India	2
Mexico	2
Spain	4

- This query groups the rows in the Customers table by the Country column and then counts how many customers belong to each country.
- The result shows the number of customers from each country: 1 from Germany, , 2 from India, 2 from Mexico, and 4 from Spain

Filter Groups Using COUNT() with GROUP BY and HAVING

We can combine the COUNT() function with HAVING to filter the results after grouping. The HAVING clause is used to specify conditions on groups, similar to the [WHERE clause](#), but for groups.

Query:

```
SELECT Country, COUNT(*) AS CustomerCount
FROM Customers
GROUP BY Country
HAVING COUNT(*) > 2;
```

Output:

Country	CustomerCount
Spain	4

- This query counts the number of customers from each country and filters out countries with fewer than 3 customers.
- The result only includes Spain, as it is the only country with more than 2 customers.

SQL SUM() Function

The SQL SUM() function is an aggregate function used to calculate the total value of a numeric column. It is widely used in reporting, financial calculations, and data analysis to quickly get overall totals.

- Returns a single total of selected numeric values.
- Works with WHERE, GROUP BY, and HAVING for conditional sums.
- Supports expressions and DISTINCT to sum unique values only.

Syntax:

```
SELECT SUM(column_name)  
FROM table_name;
```


Examples of SQL SUM() Function

In this section, we demonstrate the SUM() function using a Sales table (Product, Quantity, Price) to understand how it calculates totals and distinct sums; consider this Sales table for the below examples:

Product	Quantity	Price
Laptop	2	1000
Smartphone	5	500
Tablet	3	300
Laptop	1	1000
Smartphone	2	500

Example 1: Using SUM() with One Column

In this example, SUM() calculates the total of a single numeric column.

Query:

```
SELECT SUM(Price) AS TotalPrice  
FROM Sales;
```

TotalPrice
3300

Example 2: Using SUM() with an Expression

In this example, calculates total revenue by multiplying quantity and price.

Query:

```
SELECT SUM(Quantity * Price) AS TotalRevenue  
FROM Sales;
```

- Multiplies Quantity × Price for each row.
- Returns the total revenue as TotalRevenue.

TotalRevenue
7400

Example 3: Using SUM() with GROUP BY

In this example, SUM() is used along with GROUP BY clause. It calculates revenue separately for each product.

Query:

```
SELECT Product, SUM(Quantity * Price) AS TotalRevenue  
FROM Sales  
GROUP BY Product;
```

- Calculates revenue for each product.
- Groups rows by Product before summing.

Product	TotalRevenue
Laptop	3000
Smartphone	3500
Tablet	900

Example 4: Using SUM() with DISTINCT

In this example, SUM() is used with the DISTINCT keyword to sum only unique price values.

Query:

```
SELECT SUM(DISTINCT Price) AS SumDistinctPrice  
FROM Sales;
```

SumDistinctPrice
1800

- Adds only unique Price values.
- Returns the result as SumDistinctPrice.

Example 5: Using SUM() with HAVING

The HAVING clause can be used with GROUP BY to filter groups based on the result of the SUM() function, allowing conditions on aggregated data.

Query:

```
SELECT Product, SUM(Quantity * Price) AS TotalRevenue  
FROM Sales  
GROUP BY Product  
HAVING SUM(Quantity * Price) > 2000;
```

Product	TotalRevenue
Laptop	3000
Smartphone	3500

- Filters products after grouping.
- Shows only products with revenue greater than 2000.

SQL MAX() function

The SQL MAX() function helps quickly identify the highest value within any column, making it useful for summarizing and analyzing data trends. It is widely used in reports to find top values like peak sales, latest dates, or highest scores.

Returns the maximum value from the specified column.

Supports numeric, date, and text data types.

Skips NULL values during calculation.

Syntax:

```
SELECT MAX(column_name)  
FROM table_name;
```

Examples of SQL MAX() Function

Here, we demonstrate the usage of the MAX() function using a single sample table named Products. Consider this Products table for all the examples below:

product_id	product_name	category	price	quantity	total_sales	sale_date
1	Laptop	Electronics	45000	2	90000	2023-01-15
2	Laptop	Electronics	45000	1	45000	2023-02-01
3	Tablet	Electronics	30000	3	90000	2023-01-20
4	Tablet	Electronics	30000	2	60000	2023-02-03
5	Chair	Furniture	7000	4	28000	2023-01-18

Example 1: Find the Maximum Price in a Table

In this example, we see how the MAX() function retrieves the highest total_sales value from the Products table while ignoring NULL values.

Query:

```
SELECT MAX(total_sales) AS [Highest Total Sales]
FROM Products;
```

Output:

Highest Total Sales
90000

Example 2: Use MAX() with a Condition

In this example, we see how the MAX() function returns the highest price after filtering rows based on a specific condition.

Query:

```
SELECT MAX(price) AS [Highest Price in Electronics]  
FROM Products  
WHERE category = 'Electronics';
```

Output:

Highest Price in Electronics
45000

- Applies WHERE to include only products in the Electronics category.
- MAX() then returns the highest price within that filtered group.

Example 3: Find the Latest Sale Date

In this example, we use MAX() to get the most recent sale date from the table.

Query:

```
SELECT MAX(sale_date) AS [Latest Sale Date]
FROM Products;
```

Latest Sale Date
2023-02-03

Example 4: Using MAX() with GROUP BY

In this example, we use MAX() with GROUP BY to find the highest sale amount for each product.

Query:

```
SELECT product_name, MAX(total_sales) AS [Top Sales Amount]
FROM Products
GROUP BY product_name;
```

product_name	Top Sales Amount
Chair	28000
Laptop	90000
Tablet	90000

Example 5: Using MAX() in Subqueries

In this example, we use MAX() inside a subquery to get the product with the highest total_sales.

Query:

```
SELECT *  
FROM Products  
WHERE total_sales = (SELECT MAX(total_sales) FROM Products);
```

product_id	product_name	category	price	quantity	total_sales	sale_date
1	Laptop	Electronics	45000	2	90000	2023-01-15
3	Tablet	Electronics	30000	3	90000	2023-01-20

- The subquery returns the maximum total_sales value from Products.
- The main query fetches the full record that matches this highest value.

Example 6: Using MAX() with HAVING

In this example, we use MAX() with HAVING to filter product groups based on their highest total sales.

Query:

```
SELECT product_name, MAX(total_sales) AS HighestSale  
FROM Products  
GROUP BY product_name  
HAVING MAX(total_sales) > 50000;
```

product_name	HighestSale
Laptop	90000
Tablet	90000

- GROUP BY creates groups for each product_name and calculates their MAX(total_sales).
- HAVING filters only those products whose highest sale exceeds 50,000.

AVG() Function in SQL

The AVG() function in SQL calculates the average of a numeric column. It helps identify the central value of data by ignoring NULL entries. Overall, it is a quick way to summarize large datasets.

- Finds the average or mean using only non-null values.
- Useful for quick comparisons and basic data analysis.

Syntax:

```
SELECT AVG(column_name)  
FROM table_name;
```

Examples of SQL AVG() Function

id	student_name	subject	score
1	John Miller	Math	85
2	Emma Wilson	Science	80
3	Chris Evans	English	90
4	Olivia Brown	Math	75
5	Liam Carter	English	86
6	Sophia Turner	Science	80

Example 1: Calculating Overall Average Score

In this example, we use AVG() to get the complete overall average of all scores.

Query:

```
SELECT AVG(score) AS overall_average_score
FROM student_scores;
```

overall_average_score
82.66666666666667

Example 2: Calculating Average Score per Subject

In this example, we use `AVG()` with the `GROUP BY` clause to find the average score for each subject.

Query:

```
SELECT subject, AVG(score) AS average_score  
FROM student_scores  
GROUP BY subject;
```

- `GROUP BY` groups rows by subject.
- `AVG()` calculates the average score for each group.

subject	average_score
English	88
Math	80
Science	80

Example 3: Calculating Average Score for a Specific Subject

In this example, `AVG()` is used with a `WHERE` clause to find the average score only for Science.

Query:

```
SELECT AVG(score) AS average_science_score  
FROM student_scores  
WHERE subject = 'Science';
```

- `WHERE` filters rows to include only Science.
- `AVG()` calculates the average from the filtered data.

average_science_score
80

Example 4: Showing Only Subjects with Average Score Above 85

In this example, we use AVG() with HAVING to display only those subjects whose average score exceeds 85.

Query:

```
SELECT subject, AVG(score) AS average_score
FROM student_scores
GROUP BY subject
HAVING AVG(score) > 85;
```

subject	average_score
English	88

- HAVING filters groups based on aggregate values.
- Only subjects with an average score > 85 are shown.

Considerations

While the AVG() function is a powerful tool for data analysis, there are some considerations to keep in mind:

- **NULL Handling:** AVG() ignores NULL values, which may impact results if many entries are missing.
- **Numeric Columns Only:** The function works only on numeric data—using it on non-numeric columns causes errors.
- **Precision:** AVG() can return long decimals; round the result when cleaner output is needed.

SQL Subquery

A subquery in SQL is a query nested inside another SQL query. It allows complex filtering, aggregation and data manipulation by using the result of one query inside another.

- Filters rows in the outer query based on the results returned by the WHERE clause.
- Treats the subquery as a temporary (derived) table that can be queried like a normal table using the FROM clause.
- Filters grouped or aggregated results using values produced by the HAVING clause.

Syntax:

```
SELECT column_name  
FROM table_name  
WHERE column_name expression operator  
      (SELECT column_name FROM table_name WHERE ...);
```

Types of Subqueries

Consider the following two tables for examples:

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
James	101	New York	9876543210
Sophia	102	London	8765432109
Daniel	103	Toronto	7654321098
Emma	104	Sydney	6543210987
Oliver	105	Berlin	5432109876

EmployeeID	Name	Salary	DepartmentID
1	Alice	80000	10
2	Bob	60000	10
3	Charlie	90000	20
4	Diana	75000	20
5	Evan	50000	30

DepartmentID	DepartmentName	Location
10	HR	New York
20	Engineering	Chicago
30	Sales	New York

1. Single-Row Subquery

- A single-row subquery is a subquery that returns only one value.
- Returns exactly one row as the result.

Commonly used with comparison operators such as =, >, <

Example:

```
SELECT * FROM Employees  
WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

EmployeeID	Name	Salary	DepartmentID
3	Charlie	90000	20

2. Multi-Row Subquery

A multi-row subquery is a subquery that returns more than one value.

Returns multiple rows as the result.

Requires operators that can handle multiple values, such as IN, ANY or ALL

Example:

```
SELECT * FROM Employees
```

```
WHERE DepartmentID IN (SELECT DepartmentID FROM Departments WHERE Location = 'New York');
```

EmployeeID	Name	Salary	DepartmentID
1	Alice	80000	10
2	Bob	60000	10
5	Evan	50000	30

- Identifies all departments located in New York.
- Returns the employees who belong to any of those departments.

3. Correlated Subquery

A correlated subquery is a subquery that depends on the outer query for its values.

A dependent subquery: it references columns from the outer query.

Executed once for each row of the outer query, making it slower for large datasets.

Example:

```
SELECT e.Name, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
                  FROM Employees
                  WHERE DepartmentID = e.DepartmentID);
```

Name	Salary
Alice	80000
Charlie	90000

- Calculates the average salary within each employee's department.
- Returns employees whose salary is higher than their department's average.

Examples of Using SQL Subqueries

These examples showcase how subqueries can be used for various operations like selecting, updating, deleting or inserting data, providing insights into their syntax and functionality. Through these examples, we will understand flexibility and importance of subqueries in simplifying complex database tasks.

Consider the following two tables:

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
James	101	New York	9876543210
Sophia	102	London	8765432109
Daniel	103	Toronto	7654321098
Emma	104	Sydney	6543210987
Oliver	105	Berlin	5432109876

NAME	ROLL_NO	SECTION
Emma	104	A
Oliver	105	B
Sophia	102	A

Example 1: Fetching Data Using Subquery in WHERE Clause

This example demonstrates how to use a subquery inside the WHERE clause. The inner query retrieves roll numbers of students who belong to section 'A' and the outer query fetches their corresponding details (name, location and phone number) from the Student table.

Query:

```
SELECT NAME, LOCATION, PHONE_NUMBER
FROM Student_Info
WHERE ROLL_NO IN (
    SELECT ROLL_NO
    FROM Student_Section
    WHERE SECTION = 'A'
);
```

NAME	LOCATION	PHONE_NUMBER
Sophia	London	8765432109
Emma	Sydney	6543210987

- The subquery NAME, LOCATION and PHONE_NUMBER, WHERE SECTION = 'A' finds roll numbers of students in section A.
- The outer query then uses these roll numbers to fetch details from the Student table.
- Thus, only Sophia and Emma are returned, since they are in section A.

Example 2: Using Subquery with DELETE

In this example, we use a subquery with DELETE to remove certain rows from the Student table. Instead of hardcoding roll numbers, the subquery finds them based on conditions.

Query:

```
DELETE FROM Student
WHERE ROLL_NO IN (
    SELECT ROLL_NO FROM Student WHERE ROLL_NO <= 101 OR ROLL_NO = 201
);
```

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
Sophia	102	London	8765432109
Daniel	103	Toronto	7654321098
Emma	104	Sydney	6543210987
Oliver	105	Berlin	5432109876

- The subquery selects roll numbers 101 and 201.
- The outer query deletes students having those roll numbers.

Example 3: Using Subquery with UPDATE

Subqueries can also be used with UPDATE. In this example, we update student names to "Geeks" if their location matches the result of a subquery.

Query:

```
UPDATE Student_Info  
SET NAME = 'Geeks'  
WHERE LOCATION IN (  
    SELECT LOCATION  
    FROM Student_Info  
    WHERE LOCATION IN ('London', 'Berlin')  
);
```

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
Geeks	102	London	8765432109
Daniel	103	Toronto	7654321098
Emma	104	Sydney	6543210987
Geeks	105	Berlin	5432109876

The subquery selects locations 'London' and 'Berlin'.

The outer query updates the NAME field for students whose location matches those values.

Example 4: Simple Subquery in the FROM Clause

This example demonstrates using a subquery inside the FROM clause, where the subquery acts as a temporary (derived) table.

Query:

```
SELECT NAME, PHONE_NUMBER  
FROM (  
    SELECT NAME, PHONE_NUMBER, LOCATION  
    FROM Student  
    WHERE LOCATION LIKE 'T%'  
) AS subquery_table;
```

NAME	PHONE_NUMBER
Daniel	7654321098

- The subquery (SELECT NAME, PHONE_NUMBER, LOCATION FROM Student WHERE LOCATION LIKE 'T%') fetches students whose location starts with "T" (Toronto).
- The outer query then selects only NAME and PHONE_NUMBER from this derived table.

Example 5: Subquery with JOIN

We can also use subqueries along with JOIN to connect data across tables.

Query:

```
SELECT s.NAME, s.LOCATION, ns.SECTION
FROM Student_Info s
INNER JOIN (
    SELECT ROLL_NO, SECTION
    FROM Student_Section
    WHERE SECTION = 'A'
) ns
ON s.ROLL_NO = ns.ROLL_NO;
```

NAME	LOCATION	SECTION
Emma	Sydney	A
Geeks	London	A

- The subquery extracts roll numbers of students in section A.
- Joining this with the Student_Info table on ROLL_NO returns Emma and Sophia along with their locations and section.

3. Data Manipulation Language

A relational database can be updated with new data using data manipulation language (DML) statements. The INSERT command, for instance, is used by an application to add a new record to the database.

Command	Description
INSERT	Creates a record.
UPDATE	Modifies records.
DELETE	Deletes records.

Inserting Data into the Table :

After creating the table, you can use INSERT INTO command to add data into it. This method is used when you want to insert data into all columns of a table without specifying column names. We simply provide the values for each column, in the same order that the columns are defined in the table.

Syntax:

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

- **table_name:** name of the table where data will be inserted
- **value1, value2...** : values that correspond to each column in order.

Example: Let's see how the INSERT INTO statement works with practical examples.

```
INSERT INTO Customer (CustomerID, FirstName, LastName, Country, Age,
Phone)
VALUES
(1, 'Luca', 'Bianchi', 'Italy', 23, 'xxxxxxxxxx'),
(2, 'Aiko', 'Tanaka', 'Japan', 21, 'xxxxxxxxxx'),
(3, 'Carlos', 'Gomez', 'Spain', 24, 'xxxxxxxxxx'),
(4, 'Sofia', 'Müller', 'Germany', 22, 'xxxxxxxxxx'),
(5, 'Ethan', 'Johnson', 'USA', 25, 'xxxxxxxxxx');
```

Output :

CustomerID	FirstName	LastName	Country	Age	Phone
1	Luca	Bianchi	Italy	23	xxxxxxxxxx
2	Aiko	Tanaka	Japan	21	xxxxxxxxxx
3	Carlos	Gomez	Spain	24	xxxxxxxxxx
4	Sofia	Müller	Germany	22	xxxxxxxxxx
5	Ethan	Johnson	USA	25	xxxxxxxxxx

2. Inserting Data into Specific Columns

In some cases, you might want to insert data into only certain columns, leaving the others empty or with default values (NULL). In such cases, we can specify the column names explicitly.

Syntax

```
INSERT INTO table_name (column1, column2,  
column3, ...)  
VALUES (value1, value2, value3, ...);
```

Parameters:

- **table_name**: name of the table.
- **column1, column2..**: name of first column, second column.
- **value1, value2, value3..**: the values for each specified column of the new record.

```
INSERT INTO Customer (CustomerID, FirstName, Age,  
Phone)  
VALUES (6, Ken, 35, xxxxxxxxxx);
```

3. Inserting Data from One Table into Another Table

We can also copy data from one table into another table using the INSERT INTO SELECT statement. This is very useful when we want to move or replicate data from one table to another without manually typing all the data.

Method 1: Insert All Columns from Another Table

Inserts every column from source table into destination table

Syntax : `INSERT INTO target_table
 SELECT * FROM source_table;`

Method 2: Insert Specific Columns from Another Table

Allows inserting only selected columns from the source table.

Syntax :

```
INSERT INTO target_table (col1, col2, ...)
SELECT col1, col2, ...
FROM source_table;
```

Method 3: Insert Specific Rows Based on Condition

You can also insert specific rows based on a condition by using the WHERE clause with the SELECT statement.

Syntax :

```
INSERT INTO target_table  
SELECT * FROM source_table  
WHERE condition;
```

Example :

```
INSERT INTO Student  
SELECT * FROM OldStudent  
WHERE Age > 20;
```

SQL UPDATE Statement

The SQL UPDATE statement is used to modify existing records in a table. It allows you to change one or more column values for specific rows using the WHERE clause. Without a WHERE condition, all rows in the table will be updated.

Syntax:

```
UPDATE table_name  
SET column1 = value1, column2 = value2,...  
WHERE condition
```

- **table_name:** Name of the table you want to update.
- **SET:** The column(s) you want to update and their new values.
- **WHERE:** Filters the specific rows you want to update.

Example: First, we will CREATE a demo SQL database and table, on which we will use the UPDATE Statement command.

EmployeeID	Name	Department	Salary
1	Alice	HR	50000
2	Bob	IT	60000
3	Charlie	Finance	55000

Query:

```
UPDATE Employees  
SET Salary = 65000  
WHERE Name = 'Bob';
```

EmployeeID	Name	Department	Salary
1	Alice	HR	50000
2	Bob	IT	65000
3	Charlie	Finance	55000

Examples of SQL UPDATE Statement

Let's begin by creating a Customer table with some sample data.

Query:

```
CREATE TABLE Customer (  
  CustomerID INT PRIMARY KEY,  
  CustomerName VARCHAR(50),  
  LastName VARCHAR(50),  
  Country VARCHAR(50),  
  Age INT,  
  Phone VARCHAR(15)  
);  
  
-- Insert sample data  
INSERT INTO Customer (CustomerID, CustomerName,  
  LastName, Country, Age, Phone)  
VALUES  
  (1, 'Liam', 'Brown', 'United Kingdom', 25, '441234567890'),  
  (2, 'Sofia', 'Martinez', 'Spain', 23, '341234567890'),  
  (3, 'Akira', 'Tanaka', 'Japan', 26, '811234567890'),  
  (4, 'Hans', 'Müller', 'Germany', 27, '491234567890'),  
  (5, 'Olivia', 'Dubois', 'France', 24, '331234567890');
```

Output:

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Liam	Brown	United Kingdom	25	441234567890
2	Sofia	Martinez	Spain	23	341234567890
3	Akira	Tanaka	Japan	26	811234567890
4	Hans	Müller	Germany	27	491234567890
5	Olivia	Dubois	France	24	331234567890

Example 1: Update Single Column Using UPDATE Statement

We have a Customer table and we want to Update the CustomerName where the Age is 23.

Query :

```
UPDATE Customer
SET CustomerName = 'Isabella'
WHERE Age = 23;
```

Output:

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Liam	Brown	United Kingdom	25	441234567890
2	Isabella	Martinez	Spain	23	341234567890
3	Akira	Tanaka	Japan	26	811234567890
4	Hans	Müller	Germany	27	491234567890
5	Olivia	Dubois	France	24	331234567890

Example 2: Updating Multiple Columns using UPDATE Statement

We need to update both the CustomerName and Country for a specific CustomerID.

Query:

```
UPDATE Customer
SET CustomerName = 'John',
    Country = 'Spain'
WHERE CustomerID = 1;
```

Output:

CustomerID	CustomerName	LastName	Country	Age	Phone
1	John	Brown	Spain	25	441234567890
2	Isabella	Martinez	Spain	23	341234567890
3	Akira	Tanaka	Japan	26	811234567890
4	Hans	Müller	Germany	27	491234567890
5	Olivia	Dubois	France	24	331234567890

Example 3: Omitting WHERE Clause in UPDATE Statement

If we accidentally omit the WHERE clause, all the rows in the table will be updated, which is a common mistake. Let's update the CustomerName for every record in the table:

Query:

```
UPDATE Customer
SET CustomerName = 'ALice';
```

Output

CustomerID	CustomerName	LastName	Country	Age	Phone
1	ALice	Brown	Spain	25	441234567890
2	ALice	Martinez	Spain	23	341234567890
3	ALice	Tanaka	Japan	26	811234567890
4	ALice	Müller	Germany	27	491234567890
5	ALice	Dubois	France	24	331234567890

SQL DELETE Statement

The SQL DELETE statement is used to remove specific rows from a table while keeping the table structure intact. It is different from DROP, which deletes the entire table.

- It removes rows based on conditions.
- Retains table schema, constraints, and indexes.
- Can delete a single row or all rows.

Syntax:

```
DELETE FROM table_name  
WHERE some_condition;
```

- **Some_condition:** A condition used to filter the rows you want to delete.
- **table_name:** The name of the table from which you want to delete the rows.

Query

```
DELETE FROM Employees  
WHERE EmployeeID = 5;
```

Examples of SQL DELETE Statement

Assume we have created a table named GFG_Employee in SQL, which contains the personal details of the Employee including their id, name, email and department etc. as shown below.

```
CREATE TABLE GFG_Employees (  
id INT PRIMARY KEY,  
name VARCHAR (20) ,  
email VARCHAR (25),  
department VARCHAR(20)  
);
```

```
INSERT INTO GFG_Employees (id, name, email, department) VALUES  
(1, 'Jessie', 'jessie23@gmail.com', 'Development'),  
(2, 'Praveen', 'praveen_dagger@yahoo.com', 'HR'),  
(3, 'Bisa', 'dragonBall@gmail.com', 'Sales'),  
(4, 'Rithvik', 'msvv@hotmail.com', 'IT'),  
(5, 'Suraj', 'srjsunny@gmail.com', 'Quality Assurance'),  
(6, 'Om', 'OmShukla@yahoo.com', 'IT'),  
(7, 'Naruto', 'uzumaki@konoha.com', 'Development');
```

```
Select * From GFG_Employees
```

Output :

id	name	email	department
1	Jessie	jessie23@gmail.com	Development
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
4	Rithvik	msvv@hotmail.com	IT
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT
7	Naruto	uzumaki@konoha.com	Development

Example 1: Deleting Single Record

We can use the DELETE statement with a condition to delete a specific row from a table. The WHERE clause ensures only the intended record is removed. We can delete the records named Rithvik by using the below query:

Query:

```
DELETE FROM GFG_Employees  
WHERE NAME = 'Rithvik';
```

Output:

id	name	email	department
1	Jessie	jessie23@gmail.com	Development
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT
7	Naruto	uzumaki@konoha.com	Development

Example 2: Deleting Multiple Records

To delete multiple records, you can specify a condition that matches several rows. Let's delete the rows from the table `GFG_Employees` where the department is "Development". This will delete 2 rows (the first row and the seventh row).

Query:

```
DELETE FROM GFG_Employees  
WHERE department = 'Development';
```

Output

id	name	email	department
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT

Example 3: Delete All Records from a Table

If we need to delete all records from the table, we can omit the WHERE clause, or alternatively use the DELETE statement with an asterisk (*) to denote all rows.

Query:

```
DELETE FROM GFG_Employees;  
Or  
DELETE * FROM GFG_Employees;
```

Output:

id	name	email	department
empty			

4. Data Control language

DCL commands manage user access to the database by granting or revoking permissions. Database administrators use DCL to enforce security and control access to database objects.

Command	Description
GRANT	Gives a privilege to the user.
REVOKE	Takes back privileges granted by the user.

5. Transaction Control Language

TCL commands manage transactions in relational databases, ensuring data integrity and consistency. These commands are used to commit changes or roll back operations in case of errors.

Command	Description
COMMIT	Saves all changes made during the current transaction on a permanent basis. Some databases provide an auto-commit feature, which can be configured using settings.
ROLLBACK	Reverts changes made during the current transaction, ensuring no unwanted changes are saved.
SAVEPOINT	Sets a point within a transaction to which changes can be rolled back, allowing partial rollbacks

Rolling Back DELETE Operations

Since the DELETE statement is a DML operation, it can be rolled back when executed in a statement. If you accidentally delete records or need to repeat the process, you can use the ROLLBACK command.

Query:

```
BEGIN TRANSACTION;  
DELETE FROM GFG_Employees;  
WHERE department = 'Development';  
  
-- If needed, you can rollback the deletion  
ROLLBACK;
```

Explanation: The ROLLBACK command will undo the changes made by the DELETE statement, effectively restoring the records that were deleted during the transaction.

SQL TRANSACTIONS

An SQL transaction is a sequence of one or more SQL operations (e.g., INSERT, UPDATE, DELETE) executed as a single unit of work. Transactions ensure that either all operations succeed or none are applied, maintaining data integrity.

Key Properties of SQL Transactions: ACID

The integrity of SQL transactions is governed by the ACID properties, which guarantee reliable database transactions. These four properties work together to guarantee that the database remains consistent and reliable.

Atomicity: The outcome of a transaction can either be completely successful or completely unsuccessful. The whole transaction must be rolled back if one part of it fails.

Consistency: Transactions maintain integrity restrictions by moving the database from one valid state to another.

Isolation: Concurrent transactions are isolated from one another, assuring the accuracy of the data.

Durability: Once a transaction is committed, its modifications remain in effect even in the event of a system failure.

SQL Transaction Control Commands

In SQL, transaction control commands manage the execution of SQL operations, ensuring the integrity and reliability of database transactions. These commands help manage the start, commit, and rollback of changes made to the database. Below are the key transaction control commands in SQL, explained with syntax and examples for each.

1. BEGIN TRANSACTION Command

The BEGIN TRANSACTION command marks the beginning of a new transaction. All SQL statements that follow this command will be part of the same transaction until a COMMIT or ROLLBACK is encountered. This command doesn't make any changes to the database, it just starts the transaction.

Syntax:

```
BEGIN TRANSACTION transaction_name ;
```



```
BEGIN TRANSACTION;
```

```
-- Deduct $150 from Account A
```

```
UPDATE Accounts
```

```
SET Balance = Balance - 150
```

```
WHERE AccountID = 'A';
```

```
-- Add $150 to Account B
```

```
UPDATE Accounts
```

```
SET Balance = Balance + 150
```

```
WHERE AccountID = 'B';
```

```
-- Commit the transaction if both operations succeed
```

```
COMMIT;
```

If any error occurs, such as an issue with the UPDATE query, you can use ROLLBACK to undo all changes made during the transaction:

```
ROLLBACK;
```

This ensures that the system doesn't end up in an inconsistent state, such as deducting money from one account without adding it to another.

```
BEGIN TRANSACTION TransferFunds;
```

2. COMMIT Command

The COMMIT command is used to save all changes made during the current transaction to the database. Once a transaction is committed, the changes are permanent.

Syntax:

```
COMMIT;
```

Example

Here is the sample Student table that will be used to perform the operations in this example. This table contains basic student details such as ID, name, age, and other relevant information that will be manipulated using various transaction control commands.

Student				
Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
3	Sujit	Rohtak	9156253131	20
4	Suresh	Delhi	9156768971	18
3	Sujit	Rohtak	9156253131	20
2	Ramesh	Gurgaon	9652431543	18

Following is an example which would delete those records from the table which have age = 20 and then COMMIT the changes in the database.

```
DELETE FROM Student WHERE AGE = 20;  
COMMIT;
```

Output

Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
4	Suresh	Delhi	9156768971	18
2	Ramesh	Gurgaon	9652431543	18

3. ROLLBACK Command

The ROLLBACK command is used to undo all changes made in the current transaction. It is used when an error occurs or when the desired changes cannot be completed. The database will revert to the state it was in before the BEGIN TRANSACTION was executed.

Syntax:

```
ROLLBACK;
```

Example

Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database. In this case, the DELETE operation is undone, and the changes to the database are not saved.

```
DELETE FROM Student WHERE AGE = 20;  
ROLLBACK;
```

Output:

Student				
Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
3	Sujit	Rohtak	9156253131	20
4	Suresh	Delhi	9156768971	18
3	Sujit	Rohtak	9156253131	20
2	Ramesh	Gurgaon	9652431543	18

4. SAVEPOINT Command

A SAVEPOINT is used to create a checkpoint within a transaction. We can roll back to a specific SAVEPOINT instead of rolling back the entire transaction. This allows us to undo part of the transaction rather than the entire transaction.

Syntax:

```
SAVEPOINT SAVEPOINT_NAME;
```

Example

```
SAVEPOINT SP1;  
//Savepoint created.  
DELETE FROM Student WHERE AGE = 20;  
//deleted  
SAVEPOINT SP2;  
//Savepoint created.
```

Output:

Student				
Roll_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
3	Sujit	Rohtak	9156253131	20
4	Suresh	Delhi	9156768971	18
3	Sujit	Rohtak	9156253131	20
2	Ramesh	Gurgaon	9652431543	18

6. RELEASE SAVEPOINT Command

This command is used to remove a SAVEPOINT that we have created. Once a SAVEPOINT has been released, we can no longer use the ROLLBACK command to undo transactions performed since the last SAVEPOINT. It is used to initiate a database transaction and used to specify characteristics of the transaction that follows.

Syntax:

```
RELEASE SAVEPOINT SAVEPOINT_NAME;
```

Example

Once the savepoint SP2 is released, we can no longer roll back to it.

```
RELEASE SAVEPOINT SP2; -- Release the second savepoint.
```

Why Use Transactions in Banking?

In this case, without a transaction, you risk scenarios where money is deducted from one account but not added to the other, leaving the system in an inconsistent state. Transactions ensure that such issues are avoided by guaranteeing that both operations succeed or fail together.

Stored Procedures

SQL Stored Procedures for SQL Server

A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again. So if you have an SQL query that you write over and over again, save it as a stored procedure, and then just call it to execute it.

You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed.

Stored Procedure Syntax

```
CREATE PROCEDURE procedure_name  
(parameter1 data_type, parameter2 data_type, ...)  
AS  
BEGIN  
-- SQL statements to be executed  
END
```

Key Terms

CREATE PROCEDURE: This keyword creates the stored procedure with the given name.

@parameter1, @parameter2: These are input parameters that allow you to pass values into the stored procedure.

BEGIN...END: These keywords define the block of SQL statements that make up the procedure body.

Example of Creating a Stored Procedure

In this example, we create a stored procedure called **GetCustomersByCountry**, which accepts a **Country** parameter and returns the **CustomerName** and **ContactName** for all customers from that country. The procedure is designed to query the **Customers** table, which contains customer information, including their **names**, **contact details**, and **country**.

CustomerID	CustomerName	ContactName	Country
1	Shubham	Thakur	India
2	Aman	Chopra	Australia
3	Naveen	Tulasi	Sri Lanka
4	Aditya	Arpan	Austria
5	Nishant	Jain	Spain

By passing a country as a parameter, the stored procedure dynamically fetches the relevant customer details from the table

Note: We will need to make sure that the user account has the **necessary privileges** to create a **database**. We can try logging in as a **different user** with administrative privileges or contact the database administrator to grant the necessary privileges to our user account. If we are using a cloud-based database service, make sure that we have correctly configured the user account and its permissions


```
-- Create a stored procedure named "GetCustomersByCountry"  
CREATE PROCEDURE GetCustomersByCountry  
    @Country VARCHAR(50)  
AS  
BEGIN  
    SELECT CustomerName, ContactName  
    FROM Customers  
    WHERE Country = @Country;  
END;  
  
-- Execute the stored procedure with parameter "Sri lanka"  
EXEC GetCustomersByCountry @Country = 'Sri lanka';
```

Output :

CustomerName	Contact Name
Naveen	Tulasi

Real-World Use Cases for SQL Stored Procedures :

- **Order Processing System** In an e-commerce application, a stored procedure can automate the process of inserting new orders, updating stock levels, and generating invoices.
- **Employee Management System** A stored procedure can be used to calculate salaries for employees, deduct taxes, and generate monthly salary slips.
- **Data Validation** Use stored procedures to validate data before it's inserted into the database. For example, checking if an email address already exists before adding a new user.
- **Audit Logs** Create a stored procedure that automatically logs changes to sensitive data, such as changes to user roles or permissions, for security and auditing purposes.

SQL Triggers

A trigger is a special stored procedure in a database that automatically executes when specific events (like INSERT, UPDATE, or DELETE) occur on a table. Triggers help automate tasks, maintain data consistency, and record database activities. Each trigger is tied to a particular table and runs without manual execution.

Syntax:

```
create trigger [trigger_name]
[before | after]
{insert | update | delete}
on [table_name]
FOR EACH ROW
BEGIN
END;
```

In this explanation:

- **trigger_name:** The name of the trigger to be created
- **BEFORE | AFTER:** Specifies whether the trigger is fired before or after the triggering event (INSERT, UPDATE, DELETE).
- **{INSERT | UPDATE | DELETE}:** Specifies the operation that will activate the trigger.
- **table_name:** The name of the table the trigger is associated with.
- **FOR EACH ROW:** Indicates that the trigger is row-level, meaning it executes once for each affected row.
- **trigger_body:** The SQL statements to be executed when the trigger is fired.

Example: Automatically Track When a User Record Is Updated

Step 1: Create Main Table

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  email VARCHAR(100),  
  updated_at TIMESTAMP  
);
```

Step 2: Create Trigger

This trigger automatically updates the updated_at field whenever the user record is modified.

```
CREATE TRIGGER update_timestamp  
BEFORE UPDATE ON users  
FOR EACH ROW  
BEGIN  
  SET NEW.updated_at = CURRENT_TIMESTAMP;  
END;
```

Step 3: Insert Sample Data

```
INSERT INTO users (id, name, email) VALUES (1, 'Amit', 'amit@example.com');
```

Output

id	name	email	updated_at
1	Amit	amit@example.com	NULL (not updated yet)

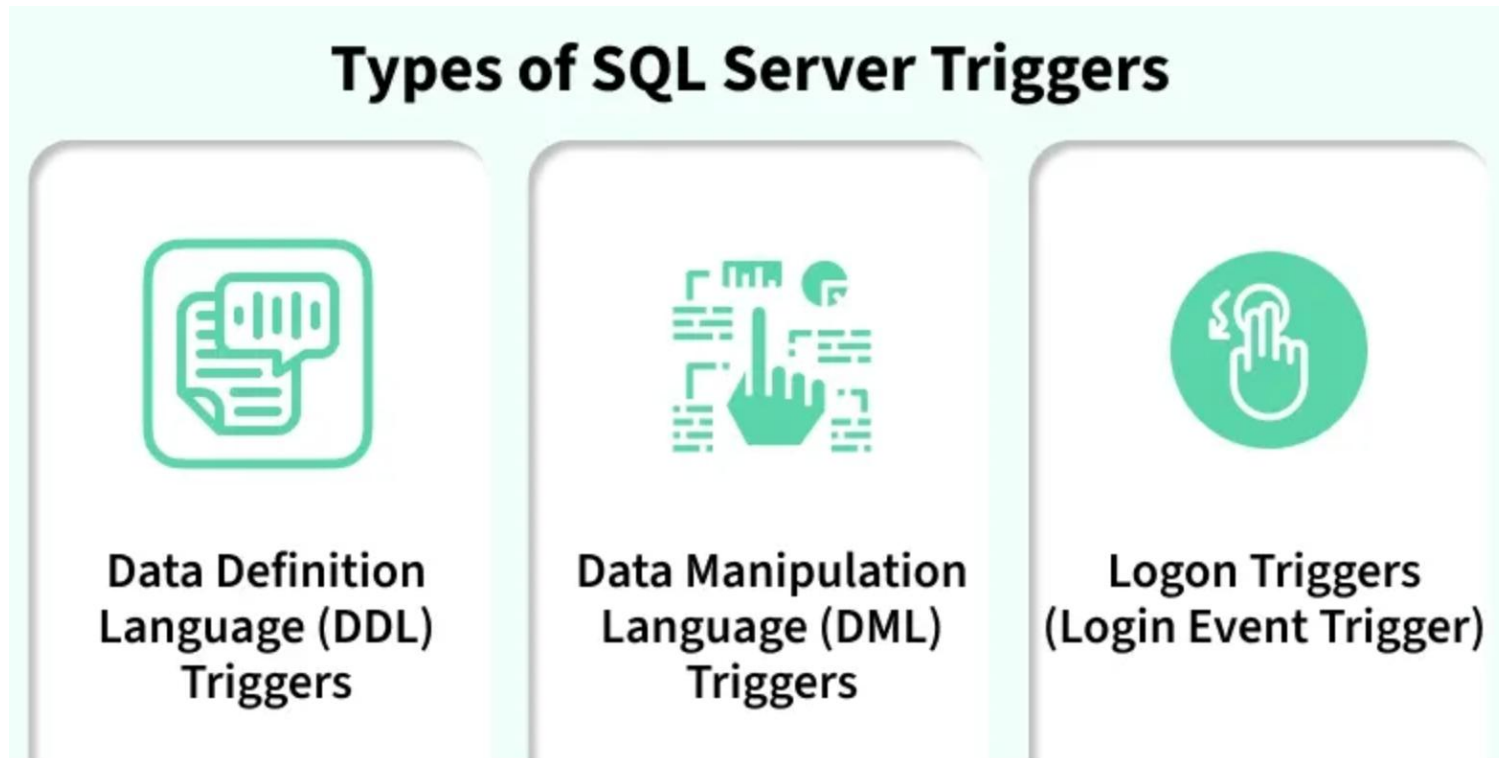
Step 4: Update a Record

```
UPDATE users SET email = 'amit_new@example.com' WHERE id = 1;
```

id	name	email	updated_at
1	Amit	amit_new@example.com	2025-10-27 12:45:23 (current timestamp auto-updated)

Types of SQL Triggers

Triggers can be categorized into different types based on the action they are associated with:



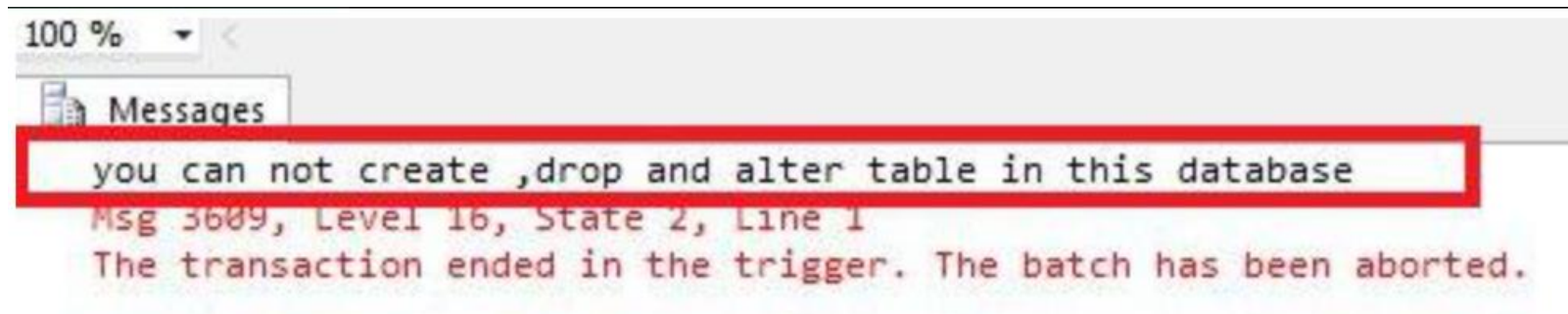
1. DDL Triggers

The Data Definition Language (DDL) command events such as Create_table, Create_view, drop_table, Drop_view, and Alter_table cause the DDL triggers to be activated. They allow us to track changes in the structure of the database. The trigger will prevent any table creation, alteration, or deletion in the database.

Example: Prevent Table Deletions :

```
CREATE TRIGGER prevent_table_creation
ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
BEGIN
PRINT 'you can not create, drop and alter table in this database';
ROLLBACK;
END;
```

Output:



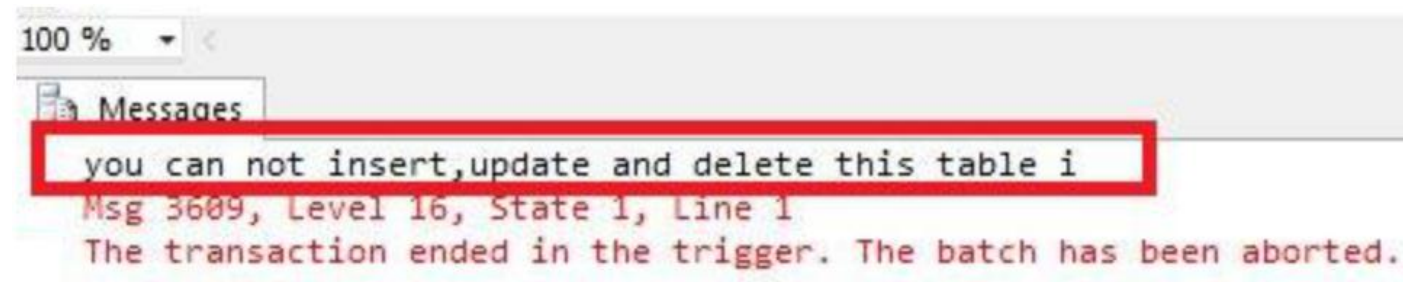
2. DML Triggers

DML triggers fire when we manipulate data with commands like INSERT, UPDATE, or DELETE. These triggers are perfect for scenarios where we need to validate data before it is inserted, log changes to a table, or cascade updates across related tables.

Example: Prevent Unauthorized Updates Let's say you want to prevent users from updating the data in a sensitive students table. We can set up a trigger to handle that:

```
CREATE TRIGGER prevent_update
ON students
FOR UPDATE, INSERT, DELETE
AS
BEGIN
RAISERROR ('You can not insert, update and delete rows in this table.', 16, 1);
END;
```

Output:



Note: ROLLBACK TRANSACTION can be included for safety, but in most cases RAISERROR itself will prevent the DML from completing.

3. Logon Triggers

Logon triggers fire in response to user logon events. They are used to monitor login activity, restrict access, or limit active sessions for a login. Messages and errors from these triggers appear in the SQL Server error log. However, they cannot handle authentication errors.

Example: Track User Logins

```
CREATE TRIGGER track_logon  
ON LOGON  
AS  
BEGIN  
PRINT 'A new user has logged in.';  
END;
```

Real-World Use Cases of SQL Triggers

1. Automatically Updating Related Tables (DML Trigger Example)

Triggers can automatically perform tasks, like updating related tables when data changes. Imagine we have a database for students, where the `student_grades` table holds individual subject grades. If the grade of a student is updated, we may also need to update the `total_scores` table.

```
CREATE TRIGGER update_student_score  
AFTER UPDATE ON student_grades  
FOR EACH ROW  
BEGIN  
  UPDATE total_scores  
  SET score = score + :new.grade  
  WHERE student_id = :new.student_id;  
END;
```

This ensures that every time a student's grade is updated, the total score in the `total_scores` table is automatically recalculated.

2. Data Validation (Before Insert Trigger Example)

Triggers can be used to validate data before it is inserted into a table, ensuring that the data follows specific business rules. For instance, we may want to ensure that the grades being inserted are within a valid range (say 0 to 100).

```
CREATE TRIGGER validate_grade  
BEFORE INSERT ON student_grades  
FOR EACH ROW  
BEGIN  
IF :new.grade < 0 OR :new.grade > 100 THEN  
RAISE_APPLICATION_ERROR(-20001, 'Invalid grade value.');  
END IF;  
END;
```

The trigger checks if the inserted grade is valid. If not, it throws an error and prevents the insertion.

Viewing and Managing Triggers in SQL

If we are working with many tables across multiple **databases**, we can use a simple query to list all available triggers in our SQL Server **instance**. This is helpful for **tracking** and **managing triggers**, especially when dealing with tables that have similar names across **databases**.

```
SELECT name, is_instead_of_trigger  
FROM sys.triggers  
WHERE type = 'TR';
```

Key Terms

- **name**: The name of the trigger.
- **is_instead_of_trigger**: Whether the trigger is an INSTEAD OF trigger.
- **type = 'TR'**: This filters the results to show only triggers.

BEFORE and AFTER Triggers

SQL triggers can be specified to run **BEFORE** or **AFTER** the triggering event.

BEFORE Triggers: These run before the action (INSERT, UPDATE, DELETE) is executed. They're great for **data validation** or modifying values before they are committed to the database.

AFTER Triggers: Execute after the SQL statement completes. Useful for logging or cascading updates to other tables.

Example: Using BEFORE Trigger for Calculations

Given Student Report Database, in which student marks assessment is recorded. In such a schema, create a trigger so that the total and percentage of specified marks are automatically inserted whenever a record is inserted. Here, a trigger will invoke before the record is inserted so BEFORE Tag can be used.

Query

```
mysql>>desc Student;
```

Output:

Field	Type	Null	Key	Default	Extra
tid	int(4)	NO	PRI	NULL	auto_increment
name	varchar(30)	YES		NULL	
subj1	int(2)	YES		NULL	
subj2	int(2)	YES		NULL	
subj3	int(2)	YES		NULL	
total	int(3)	YES		NULL	
per	int(3)	YES		NULL	

Below SQL statement will create a trigger in the student database in which whenever subjects marks are entered, before inserting this data into the database, the trigger will compute those two values and insert them with the entered values. In this way, triggers can be created and executed in the databases.

```
CREATE TRIGGER stud_marks
BEFORE INSERT ON Student
FOR EACH ROW
SET NEW.total = NEW.subj1 + NEW.subj2 + NEW.subj3,
    NEW.per = (NEW.subj1 + NEW.subj2 + NEW.subj3) * 60 / 100;
```

Output:

Trigger	Event	Table	Timing	Created	Status
stud_marks	BEFORE	Student	BEFORE INSERT	2023-05-02 00:00:00	ACTIVE

Practice resources :

<https://www.practice-sql.com/sql-practice>

<https://www.sql-practice.com/>

Q&A